Fail-Fast vs Weakly Consistent Iterators

9 min read Updated 28 Jul 2026 Topic 22 of 65

Bookmark Mark done

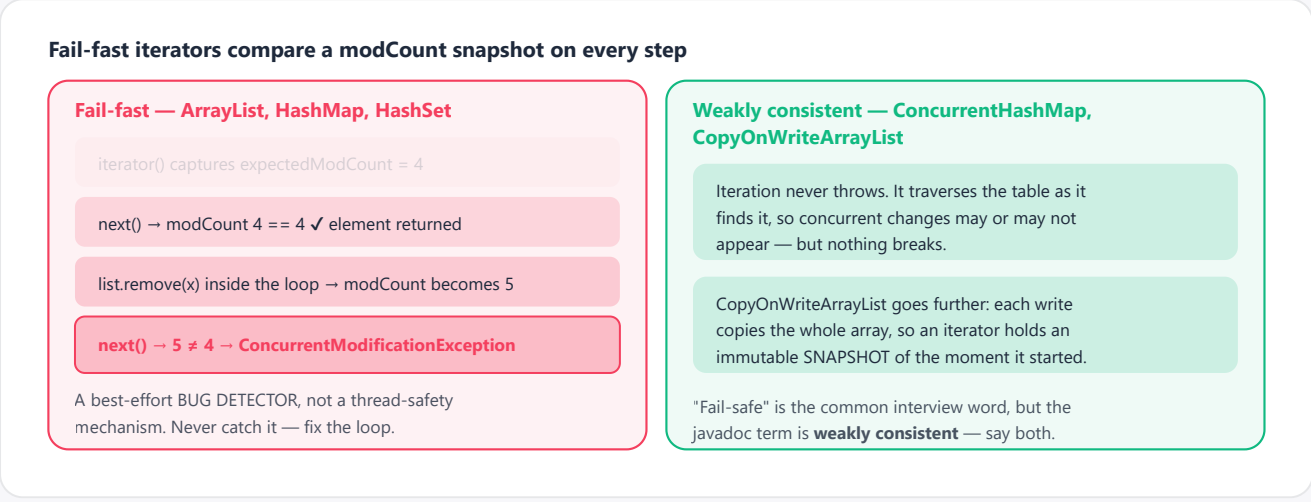
Why it matters

`ConcurrentModificationException` is one of the first runtime exceptions every Java developer meets, and one of the most misunderstood — the name says "concurrent", so people assume threads, when the overwhelmingly common cause is a single thread modifying a collection inside its own loop. Understanding the actual mechanism tells you why it happens, why you must never catch it, and why the "fail-safe" alternative is not really safe in the way the name suggests.

The concept, simply

Fail-fast iterators detect that their collection was structurally modified while they were iterating, and throw immediately. The standard collections — `ArrayList`, `HashMap`, `HashSet` — all behave this way.

Weakly consistent iterators (commonly called "fail-safe" in interviews) never throw. They iterate over whatever state they can see, and concurrent changes may or may not be reflected. The concurrent collections behave this way.



How it works

The implementation is genuinely simple, and being able to describe it is the whole answer:

JAVA Copy

```java
// Inside ArrayList, paraphrased:
protected transient int modCount = 0;     // bumped by every STRUCTURAL cha

private class Itr implements Iterator<E> {
    int expectedModCount = modCount;      // snapshot taken at creation

    public E next() {
        checkForComodification();
        // … return the element …
    }

    final void checkForComodification() {
        if (modCount ≠ expectedModCount) throw new ConcurrentModification
    }
}
```

"Structural" is the key word. Adding or removing elements changes the structure and bumps `modCount`. Replacing an element's value does not — `list.set(i, x)` and `map.put(existingKey, newValue)` are both fine during iteration. Only insertion and removal are structural.

It is explicitly best-effort. The javadoc is unusually blunt: the check is unsynchronised, so in a genuinely concurrent scenario the exception may or may not be thrown. *"Fail-fast iterators throw `ConcurrentModificationException` on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness."* It exists to surface bugs during development, not to provide a guarantee.

Code examples

JAVA Copy

```java
List<String> list = new ArrayList<>(List.of("a", "b", "c"));

// ✗ ConcurrentModificationException — the for-each loop is an iterator.
for (String s : list) {
    if (s.equals("b")) list.remove(s);
}

// ✓ 1. removeIf — clearest for a predicate.
list.removeIf(s -> s.equals("b"));

// ✓ 2. Explicit iterator — it updates expectedModCount for you.
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) it.remove();
}

// ✓ 3. Iterate a copy, modify the original.
for (String s : new ArrayList<>(list)) {
    if (s.equals("b")) list.remove(s);
}

// ✓ 4. Collect first, remove after the loop.
List<String> doomed = list.stream().filter(s -> s.equals("b")).toList();
list.removeAll(doomed);
```

The one that looks like it should work and does not:

JAVA Copy

```java
List<String> list = new ArrayList<>(List.of("a", "b", "c"));

for (String s : list) {
    if (s.equals("b")) {
        list.remove(s);
        break;                  // ← seems safe, and usually IS…
    }
}
// …because hasNext() is size-based: after removing the second of three
```

```
// elements, cursor(2) == size(2), so hasNext() returns false and next() —
// the check lives — is never called. Removing the
// SECOND-TO-LAST element silently skips the exception AND the last elemen
// It is not a safe pattern; it is an accident of the implementation.
```

Maps, where the fix is to iterate the right view:

JAVA Copy

```
Map<String, Integer> map = new HashMap<>(Map.of("a", 1, "b", 2, "c", 3));

// ✗ throws
for (String k : map.keySet()) if (map.get(k) < 2) map.remove(k);

// ✓ entrySet iterator supports removal
Iterator<Map.Entry<String, Integer>> it = map.entrySet().iterator();
while (it.hasNext()) if (it.next().getValue() < 2) it.remove();

// ✓ one-liner
map.entrySet().removeIf(e -> e.getValue() < 2);
map.values().removeIf(v -> v < 2);      // views write through to the map

// ✓ Replacing values during iteration is NOT structural — this is legal:
map.replaceAll((k, v) -> v * 2);
```

And the weakly-consistent contrast:

JAVA Copy

```
Map<String, Integer> safe = new ConcurrentHashMap<>(Map.of("a", 1, "b", 2)
for (String k : safe.keySet()) {
    safe.remove(k);              // ✓ no exception — ever
    safe.put("new-" + k, 0);     // may or may not appear in this iteratio
}
// Guarantees: never throws, each element returned at most once, reflects
// map at some point at or since the iterator was created. What it does NO
// give you is a consistent snapshot.

// CopyOnWriteArrayList is the stronger case — a true immutable snapshot:
List<String> cow = new CopyOnWriteArrayList<>(List.of("a", "b"));
for (String s : cow) {
```

```
cow.add("c");                    // ✓ no exception, and NEVER seen by this
                                 // the iterator holds the old array
// Its iterator also does not support remove() — it would be meaningless
// against a snapshot. It throws UnsupportedOperationException.
```

Under the hood

Why not just make the iterator handle it? Because there is no sensible answer to "what should happen next" after the underlying array has shifted or the table has resized. If element 2 was removed, should the iterator return element 3 or skip it? Should an element inserted before the cursor be revisited? Rather than pick silently and produce quietly wrong results, the designers chose to fail loudly. That is a design principle worth naming in an interview: **a clear exception beats a plausible wrong answer.**

The single-threaded misnomer. The name mentions concurrency, but no threads are needed — one thread modifying a collection during its own loop is the common cause by a wide margin. Java 9 acknowledged this indirectly by improving the message on some collections, but the name is stuck.

Streams have the same problem, with less obvious symptoms. Modifying the source collection during a stream pipeline gives undefined behaviour: it may throw `ConcurrentModificationException`, or silently produce wrong results. The `forEach` methods on collections carry the same restriction. It is the same rule wearing newer syntax.

Common mistakes

- **Catching `ConcurrentModificationException`.** It reports a bug in your loop. Fix the loop.
- **Assuming it means threads.** Usually it is one thread modifying inside its own iteration.
- **Relying on the `break` trick.** It works only for specific positions and silently skips elements otherwise.
- **Thinking "fail-safe" means safe.** It means "does not throw" — you may still read a mix of old and new state.

- **Expecting the exception to be guaranteed.** It is explicitly best-effort and unsynchronised.
- **Calling `remove()` on a `CopyOnWriteArrayList` iterator.** Unsupported — the snapshot is immutable.
- **Modifying the source during a stream.** Undefined behaviour, sometimes silent.
- **Using a concurrent collection purely to silence the exception.** That hides a design problem rather than fixing it.

Interview lens

Q: "What is a fail-fast iterator?"

A: One that detects structural modification of its collection during iteration and throws `ConcurrentModificationException` immediately. It works with a `modCount` field the collection bumps on every add or remove; the iterator snapshots it at creation and compares on every `next()`. `ArrayList`, `HashMap` and `HashSet` all behave this way.

Q: "Fail-fast vs fail-safe?"

A: Fail-fast throws on detected modification — the standard collections. Fail-safe, which the javadoc actually calls *weakly consistent*, never throws: it iterates whatever state it can see, so concurrent changes may or may not appear. That is the concurrent collections. `CopyOnWriteArrayList` is the strongest form, iterating a genuine immutable snapshot because every write copies the array.

Q: "Does this require multiple threads?"

A: No, and that is the common misconception. The usual cause is a single thread calling `list.remove( ... )` inside a for-each loop over the same list. The name is misleading — the check is just a counter comparison and does not care where the modification came from.

A: `removeIf` for a predicate; an explicit `Iterator` with `it.remove()`, which updates the expected count; iterate a copy and modify the original; or collect the doomed elements and remove them after the loop. For maps, `entrySet().removeIf( ... )` or the `entrySet` iterator's `remove()`.

Q: "Is a `set()` during iteration safe?"

A: Yes — only *structural* modification bumps `modCount`. Replacing an element's value with `list.set(i, x)`, or a `map.put` on a key that already exists, does not change the structure and does not trigger the check. Adding or removing does.

Q: "Should you ever catch it?"

A: No. It signals a bug in the iteration logic, not a recoverable condition, and the javadoc explicitly says it is thrown on a best-effort basis and must not be depended upon for correctness. Catching it papers over a loop that is already doing something undefined.

Cheat-card

- Mechanism: `modCount` vs the iterator's `expectedModCount`, checked on every `next()`.
- Only **structural** changes count — add/remove yes, `set`/value-replace no.
- **No threads required** — usually one thread modifying inside its own loop.
- Best-effort by design. **Never catch it.**
- Fixes: `removeIf` · `Iterator.remove()` · iterate a copy · collect then remove.
- Maps: `entrySet().removeIf( ... )`; `keySet`/`values` are write-through views.
- Weakly consistent = never throws, at-most-once, no snapshot guarantee.
- `CopyOnWriteArrayList` = true snapshot; its iterator's `remove()` is **unsupported**.

QUICK CHECK — no marks, just for you.

- [ ] A modCount counter on the collection, snapshotted by the iterator at creation and compared on every next() call
- [ ] A lock timeout inside the iterator
- [ ] A hash of the collection's contents, recomputed on each iteration step
- [ ] The JVM detects concurrent access to the collection from two threads

Check answer

2. Which operation during iteration will NOT trigger the fail-fast check?

- [ ] map.remove(k)
- [ ] list.set(i, x) — replacing an element is not a STRUCTURAL modification
- [ ] list.add(x)
- [ ] list.remove(x)

Check answer

3. Should you catch ConcurrentModificationException?

- [ ] Yes, and retry the iteration
- [ ] Yes, if the collection is shared between threads
- [ ] No — it reports a bug in the iteration logic, and the javadoc explicitly states it is thrown on a best-effort basis and must not be relied upon for correctness
- [ ] Only in production code, not in tests

Previous
ConcurrentHashMap and Concurrent Collections

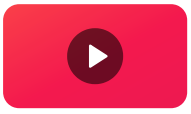
Next
Generics and Type Erasure

LEARN MORE

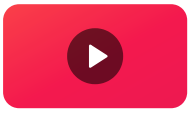
ARTICLES & DOCS

- Iterator and fail-fast behaviour — API docs
  docs.oracle.com
- ConcurrentModificationException — API docs
  docs.oracle.com

VIDEOS



- "fail fast vs fail safe" — video explainers
  YouTube



- fail fast vs fail safe interview questions
  YouTube

Hand-picked references for this lesson.